

3.4 | OPERATING SYSTEM SERVICES FOR PROCESS MANAGEMENT

This section describes the **run time** services (**system calls**) provided by kernels of multiprogramming operating system for process management. By run-time services, we mean the predefined system calls that may be used by user processes, either directly OR through a supervisor calls embedded in user's code, OR indirectly through commands typed at the terminal and translated into operating system calls by the console-monitor routine.

Although, multiprogramming systems often differ in this design and their objectives. Their innermost kernel layers exhibit similarity in the type and range of the process management primitives that they provide. The parameter details may vary from system to system but the functions provided by the total collection of operating system calls are very similar. This means that there is a strong agreement among the operating system designers, as to what constitute a minimal set of functions and services that are necessary to support process management in a multitasking environment.

Our aim in this section is to identify and describe the functions belonging to that minimal set. Other services that are basic functions on some systems may be implemented from a combination of ones described in this section. In other words, here we are interested in **operating system kernel primitives** only.

For the time being, operating system calls should be regarded as a parameterized procedure calls that are in some way predefined and accessible from systems programming languages for a particular installation of a system.

Subsequent sections describe the following process management operating system calls; process creation, process termination, FORK/JOIN, ABORT, SUSPEND/RESUME, DELAY, GET-ATTRIBUTE and CHANGE-PRIORITY.

3.4.1 Process Creation

A process may create several new process, through a **create-process** system call during the time of its execution. The creating process is called a **parent process** while the new

processes that are created are called **children** of that process. Each of these new processes may further create other process, forming an hierarchy of processes (Figure 3.6).

As we know that a process will need certain resources (such as **CPU, memory, files, input/output devices**) to accomplish its task. When a process creates a subprocess, that subprocess may be able to obtain its resources directly from the OS, or it may be constrained to a subset of the resources of the parent process. The parent may partition its resources among its children, OR it may share some resources among its several children. When a process creates a new process, **two** possibilities exist in terms of **execution**:

- (i) The parent continues to execute concurrently with its children.
- (ii) The parent waits until some or all of its children have terminated.

There are also **two** possibilities in terms of the **address space** of the new process :

- (i) The child process is a duplicate of the parent process.
- (ii) The child process has a program loaded into it.

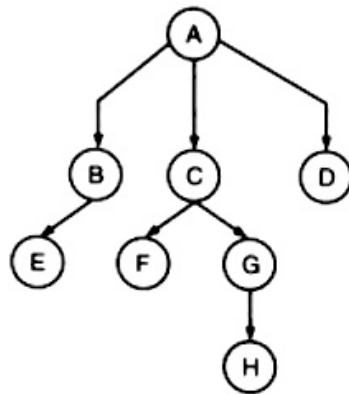


Fig. 3.6 Process hierarchy

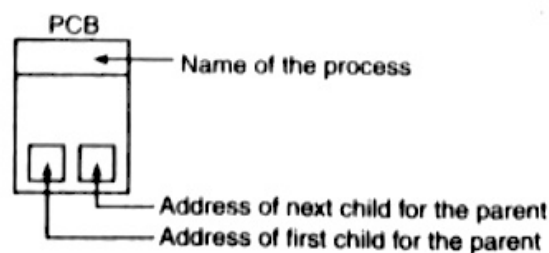
Now, the **question** arises that how this process hierarchy normally implemented by the operating system?

The OS maintains a separate pointer chain to link the related PCBs to represent the process hierarchy. Each PCB has some additional pointer fields such as:

- (i) Pointer to PCB of the first child process
- (ii) Pointer to PCB of the last child process
- (iii) Pointer to PCB of next twin process for same parent process
- (iv) Pointer to PCB of previous twin process for same parent
- (v) Pointer to PCB of parent process

Using above pointer fields for process hierarchy we get the picture as depicted in figure 3.7.

A shaded box represent a null value in the pointer i.e., end of the respective chain.



3.5 | THREADS

Referring to the concept of process switching (section 3.3.4) described with the figure 3.5 we see that at a single instant of time only one process either P_0 or P_1 is in execution state. When P_0 is executing P_1 is in idle state and when P_1 is executing, P_0 is in idle state. While at the time of switching between the two processes, the system does no useful work i.e., P_0 and P_1 both are in idle state. This situation, leads to reduce the efficiency of our system.

Even when we move from a uniprocessor system to multiprocessor systems, where multiple sets of registers exists and more switching is required then this problem will be increased.

For the solution of this problem under such situation, a new concept called as "**Threads**" are used.

Threads represent a software approach to improving performance of operating systems by reducing the overhead of process switching. A thread has some of the characteristics of a process, but it is possible to have several threads sharing the same memory space.

A thread, sometimes called a **light weight process** (LWP) with a reduced state. A thread is a basic unit of CPU utilization.

It comprises a thread ID, a program counter, a register set, and a stack. State reduction is achieved by having a group of related threads belonging to the same process and sharing its code section, data section, and other OS resources such as memory and files. In thread-based systems, threads take over the roll of processes as the smallest individual unit of scheduling. In such systems, the process OR task serves as an environment for execution of threads. The process thus becomes a unit of resource ownership, such as memory and files, for a collection of threads. If a process has multiple threads, it can be able to perform

more than one task, at a single instant of time. Figure 3.9 illustrates the difference between a single-thread and a multithreaded process.

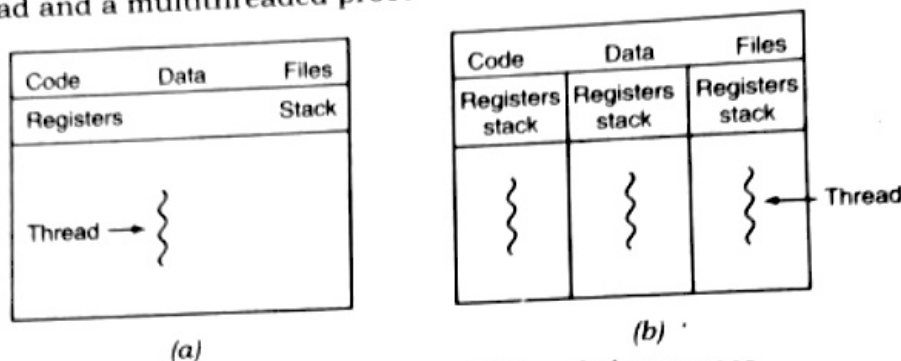


Fig. 3.9 Single and multithreaded processes

In systems with threads, a process with exactly one thread (Fig. 3.9(a)) is equivalent to a classical process. Each thread belongs to exactly one process and no thread can exist outside a process. Processes are state in nature and only threads can be scheduled for execution. Typically each thread represents a separate flow of control and is characterized by its own stack and hardware state (Figure 3.9). Since all resources other than the processor are managed by the enclosing process, so switching between related threads (threads belonging to same process) is fast and efficient. However, switching between different processes incurs the full process switch with its associated overhead.

Threads are a convenient mechanism for exploiting concurrency with in an application.

EXAMPLE 3.3 This example illustrates the concept of multithreading (or multitasking).

Many software packages that run on modern desktop PCs are multithreaded. An application is implemented as a separate process with several threads of control. A **web-browser** might have one thread to display images OR text while other thread retrieves data from the network. A **word-processor** (viz. msword) may have a thread for displaying graphics, another thread for reading the character entered by user through keyboard, and a third thread for performing spelling and grammar checking in the background.

3.5.1 Why Multithreading?

In some situations, a single application is required to perform several similar task such as, a web server accepts client requests for web pages, images, sounds, graphics etc. A busy web server may have several clients concurrently accessing it. So, if the web-server runs on traditional single-thread process, it would be able to service only one client at a time. So, the client have to wait for an enormous time to provide its server to other client. One solution of this problem can be thought by the creation of new process (illustrated in section 3.4.1). But this method is very heavy weight. (In fact, this method was in common use before the introduction threads). If the new process will perform the same task as the existing process, why incur all that overhead? It is generally more efficient for one process that contains multiple threads to serve the same purpose. This approach would multithread the web-server process. Now, the serve will create a separate thread that will listen the client's request. When a request is made, it will create another thread to service the request (instead of creating a new process).

3.5.2 Benefits of Multithreading

- **Responsiveness** : An interactive application of multithreading allows a program to run even if part of it is **blocked** OR is performing a lengthy operation i.e., it reduces the overhead and increase the responsiveness of user.
- **Low overhead** : Thread state consists only of the state of computation. Resource allocation state and communication state is not a part of thread state, which leads to low switching overhead.
- **Speed-up** : Concurrency within a process can be realized by creating multiple threads in it. This technique speed up the execution of an application in both uniprocessor and multiprocessor systems.
- **Efficient communication** : Threads of a process can communicate with one-another through shared address space (i.e., same data space), thus avoiding the overhead of system calls for communication.
- **Resource sharing** : By default, threads share the memory and the resources of the process to which they belong. The benefit of code sharing allows an application to have several different threads of activity all within the same address space.
- **Utilization in multiprocessor systems** : The benefits of multithreading can be greatly increased in a multiprocessor architecture, where each thread may be running in parallel on a different processor. A single-threaded process can only run on one processor, no matter how many processor are available in the system. Multithreading in a multiprocessor machine increases concurrency. In a uniprocessor system, CPU generally moves between each thread so quickly as to create an illusion of parallelism, but in reality only one thread is running at a time i.e., they may be **concurrent** but not **parallel**.

3.5.3 Implementation of Threads

Threads are implemented in different ways. The main difference is that how much the kernel and the application program know about the threads. These differences lead to different implications for overhead and concurrency in an application program.

In the present section, we discuss three methods of implementing threads. These are:

- Kernel-level threads
- User-level threads
- Hybrid threads

Experiments have shown that switching between kernel-level threads of a process is over 10 times faster than switching between processes; and switching between user-level threads is 100 times faster than switching between processes. Kernel-level provides better parallelism and speed-up in **multiprocessor systems**.

3.5.3.1 Kernel-level threads

A kernel-level thread is implemented by the **kernel**. Hence, creation and termination of kernel-level threads, and checking their status is performed through **system calls** analogous to those discussed in sections 3.4.1 and 3.4.2 (in process creation and termination). Figure 3.10 shows that how the kernel handles the kernel-level threads.

When a process makes a **create-thread** system call, the kernel assigns an id to it, and allocates a thread control block (TCB). TCB contains a pointer to the PCB of the process. When an event occurs, the kernel saves the CPU state of the interrupted thread in its TCB. After event handling, the scheduler considers TCB of all threads and selects one ready thread. The dispatcher uses the PCB pointer in its TCB to check if the selected thread belongs to a different process than the interrupted thread. If so, it saves the context of the process to which the interrupted thread belongs, and loads the context of the process to which the selected thread belongs. It then dispatches the selected thread. Actions to save and load the process context are not necessary, if both threads belong to the same process. This feature reduces the switching overhead.

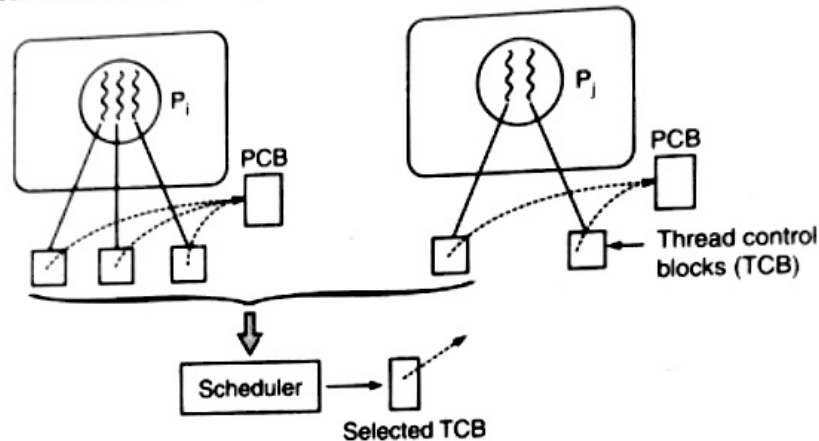


Fig. 3.10 Scheduling of kernel-level threads.

- **Advantages :** A kernel-level thread is like a process except that it has a smaller amount of state information. The similarity between process and thread is convenient for programmers—here programming for threads is same as programming for processes. In a multiprocessor system, kernel-level threads provide parallelism i.e., several threads belonging to a process can be scheduled simultaneously, which is not possible using the user-level threads described in the next section. So it provides better computation speed up than user-level threads.
- **Disadvantages :** However, handling threads like processes has its disadvantages too. Switching between threads is performed by the kernel, as a result of event handling. Hence, it incurs the overhead of event handling, even if the interrupted thread and the selected thread belongs to the same process. This feature limits the savings in the switching overhead.

3.5.3.2 User-Level Threads

User-level threads are implemented by a **thread library**, which is linked to the code of the process. The library sets up the thread implementation arrangement shown in figure 3.11 without using the kernel, and interleaves operation of threads in the process. Thus, the kernel is not aware of presence of user-level threads in a process, as it sees only the process. The scheduler considers PCBs and selects a ready process while the dispatcher dispatches it. Most OSs implement the **pthread** application program interface (API) provided in the IEEE Posix standard in this manner.

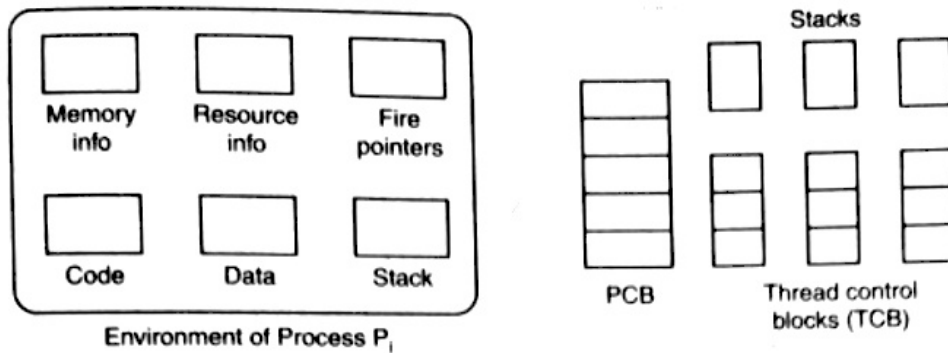


Fig. 3.11 Implementation of threads in process P_L

An overview of creation and operation of user-level threads:

A process uses the library function **create-thread** to create a new thread. The library function creates a TCB for the new thread and starts considering the new thread for scheduling. When the thread in running state uses a library function to synchronize its functioning with other threads, the library function performs 'scheduling' and switches to another thread of the process. The kernel is not able to see the switching between two threads in a process it believes that the process is continuously in operation. If the thread library cannot find a ready thread in the process, it makes a "**block me**" system call. Then the kernel now block the process. The process will be unblocked when some event activities one of its threads and will resume execution of the thread library function, which will perform scheduling and switch to the execution of the newly activated thread.

Figure 3.12 shows the scheduling of user-level threads. The thread library code is a part of each process. It performs scheduling to select a thread, and organizes its execution. We can view this operation as mapping of the TCB of the selected thread into the PCB of the process.

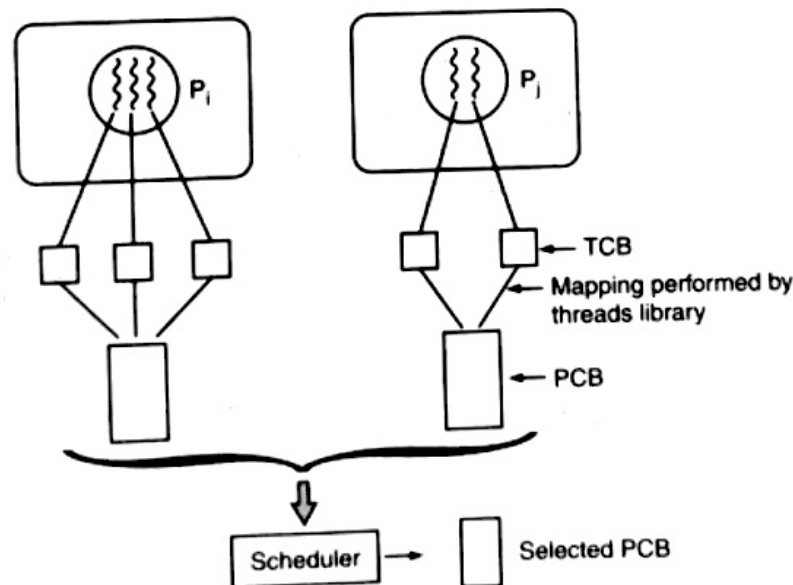


Fig. 3.12 Scheduling of user-level threads

The thread library uses information in the TCBs to decide which thread should operate. To dispatch the thread, CPU state of thread should become CPU state of process and process stack pointer should point on thread's stack. Since the thread library is a part of a process, the CPU is in the **non-privileged mode**. Hence a thread cannot be dispatched

by loading new information into the **PSW**. The thread library has to use non-privileged instructions to change the PSW contents, so it loads the address of the thread's stack into the stack address register and executes a branch instruction to transfer control to the next instruction of the thread.

Advantages: Since thread scheduling is implemented by thread library, so thread switching overhead is smaller than kernel-level threads. This arrangement enables each process to use a scheduling policy that is best suited to it. A process implementing a real time application may use priority scheduling of its threads, while process implementing a multithreaded server may perform R-R scheduling of its threads.

Disadvantages: Managing threads without the involvement of kernel has few drawbacks also such as:

1. Kernel does not know the distinction between a thread and a process, so if a thread wants to be blocked, the kernel will block its parent process. As a result, all the threads of the process get blocked until the cause of **blocking** is removed.
2. Since kernel schedules a process, and thread library schedules the thread within a process, so at most one of the thread of a process is in operation at any time. Thus, user-level threads cannot provide parallelism and the concurrency provided by them is seriously impaired if a thread makes a system call that leads to blocking.

4.1 | TYPES OF SCHEDULERS

Generally, there are three types of schedulers, which may coexist in a complex operating system:

- Long term scheduler
- Medium term scheduler
- Short term scheduler.

Fig. 4.1 shows the possible traversal paths of jobs and programs through the components and queues, in a computer system.

As shown in figure 4.1 a submitted batch job joins the batch queue while waiting to be processed by the long term scheduler. Once, scheduled for execution, processes spawned by the batch job, enter the ready queue to await the processor allocation by the short term scheduler. After becoming suspended in waiting queue, the running process may be removed from memory and swapped out to secondary storage. Such processes are subsequently admitted to main memory by the medium term scheduler in order to be considered for execution by the short-term scheduler.

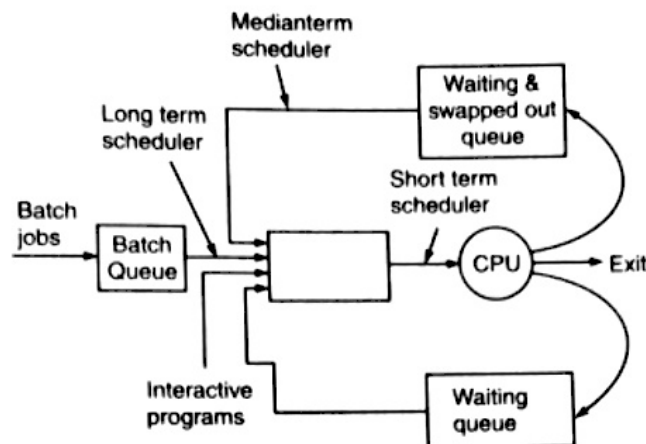


Fig. 4.1 Scheduler

4.2 OBJECTIVES OF SCHEDULING

There are many objectives that must be considered in the design of scheduling discipline such as:

- (i) **Fairness:** Fairness can be reflected by treating all the processes same and no process should suffer indefinite postponement.
- (ii) **Throughput:** It must attempt to service the largest possible number of processes per unit time.
- (iii) **Predictable:** A given job should run in about the same amount of time and at the same cost irrespective of the load on the system.
- (iv) **Overhead:** A certain portion of system resources invested as overhead can greatly improve overall performance of the system.
- (v) **Resources:** Scheduling mechanism should keep the resources of the system busy.
- (vi) **Indefinite Postponement:** Avoiding indefinite postponement of any process so that each and every process is executed in a certain amount of time.
- (vii) **Priorities:** Scheduling mechanism should favour the high priority processes.

4.3 SCHEDULING CRITERIA

Many criteria have been suggested for comparison of CPU scheduling algorithms. The characteristics used for comparison can make a substantial difference in the determination of the best algorithm. The criteria includes the following:

• CPU Utilization

We have to keep the CPU as busy as possible. It may range from 0 to 100%. In a real system it should range from 40 – 90% for lightly loaded and heavily loaded systems.

• Throughput

It is the measure of work in terms of number of processes completed per unit time. For long processes, this rate may be 1 process per hour; for short transactions, throughput may be 10 process per second.

- **Turnaround Time**

It is the sum of time-periods spent in waiting to get into memory, waiting in ready queue, executing on the CPU and doing I/O time.
i.e., $\text{turn around time} = \text{time of completion of job} - \text{time of submission of job}$

- **Waiting Time**

The CPU scheduling algorithms does not affect the amount of time during which a process executes or does I/O because they are fixed for a particular system. It affects only the amount of time that a process spends waiting in a ready queue. Waiting time is the sum of time-periods spent waiting in ready queue.

- **Response Time**

It is the amount of time, it takes to start responding, not the time taken to output that response.

4.4 | TYPES OF SCHEDULING

Nonpreemptive and preemptive scheduling schemes:

CPU scheduling decisions may take place under the following four circumstances:

- When a process switches from the running state to the waiting state (for example, input/output wait).
- When a process switches from the running state to the ready state (for example, when an interrupt occurs).
- When a process switches from the waiting state to the ready state (for example, completion of input/output).
- When a process terminates.

In circumstances (i) and (iv) there is no choice in terms scheduling. A new process (if one exists in the ready queue) must be selected for execution. When scheduling takes place only under (i) and (iv) circumstances, we say the scheduling scheme is **Nonpreemptive**.

However, in circumstances (ii) and (iii) there is a choice in terms of scheduling i.e., a process may or may not be selected for its execution, this type of scheduling is called **preemptive**.

Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state i.e. when a process starts its execution under nonpreemptive scheduling then its execution is completed until the process in execution completes its burst time without any interruption or preemption. Also it does not require any special hardware such as "timer" needed for preemptive scheduling.

While in preemptive scheduling, execution of a process is preempted before the completion of burst time of the process and any other process may starts its execution whose priority is higher than the first arrived process in the system.

4.5 | SCHEDULING ALGORITHMS

The scheduling mechanism describe in this section is used by any of the above described there types of schedulers: long-term, medium-term or short term. But, some algorithms are better suited to the needs of a particular type of scheduler.

CPU scheduling deals with the problem of deciding which of the process in the **ready queue** is to be allocated the CPU. Now we describe several of the many CPU scheduling algorithm that exist:

4.5.1 First-Come, First-Served (FCFS) Scheduling Algorithm

FCFS is the simplest of all scheduling algorithms. The key concept of this algorithm is "allocated the CPU in the order in which the processes arrive." That is the process that requests the CPU first is allocated the CPU first. The implementation of FCFS policy is managed with a FIFO (First-in-first out) queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue. The code for FCFS scheduling is simple to write and understand.

FCFS Scheduling Algorithm is Non-preemptive

Once the CPU has been given to a process, that process keeps the CPU until it releases the CPU either by terminating or by requesting I/O devices. The FCFS algorithm is suitable for Batch operating system. The average waiting time under FCFS policy is often quite long especially if requests of short CPU burst waits behind the long ones. FCFS is not suitable for time-sharing systems where it is important that each user should get the CPU for an equal amount of arrival time.

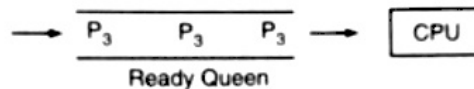


Fig. 4.2 FCFS Scheduling

EXAMPLE 4.1 Consider the following set of processes having their burst time mentioned in milliseconds. CPU-burst time indicates that for how much time, the process needs the CPU.

Process	Burst Time (ms)
P_1	20
P_2	7
P_3	5

Calculate the average waiting time if the processes arrive in the order of:

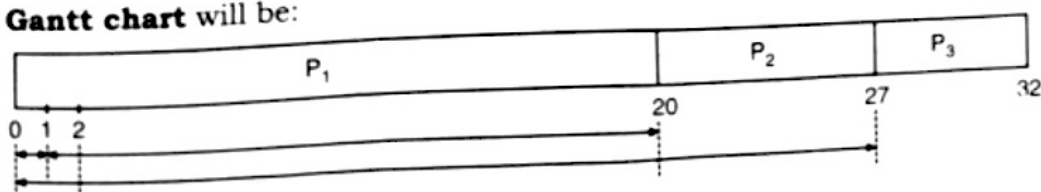
- (i) P_1, P_2, P_3 (ii) P_2, P_3, P_1
 (iii) P_3, P_1, P_2 (iv) P_3, P_2, P_1

Solution: (i) If processes arrive in order P_1, P_2, P_3 let us assume at 0 ms, 1 ms and 2 ms respectively.

Process	Burst time	Arrival time
P_1	20	0
P_2	7	1
P_3	5	2

Since, FCFS is a non-preemptive algorithm; so when a process execution is started it is not preempted until its execution is completed.

The **Gantt chart** will be:



Waiting time for process $P_1 = 0$ ms

Waiting time for process $P_2 = 20$ ms - 1 ms = 19 ms

Waiting time for process $P_3 = 27$ ms - 2 ms = 25 ms

$$\therefore \text{Average waiting time} = \frac{0 + 19 + 25}{3} \text{ ms}$$

$$= \frac{44}{3} \text{ ms} = 14.67 \text{ ms.}$$

Consider a case, if the processes P_1, P_2 and P_3 arrives at the same time say at 0 ms in the system, in the order of P_1, P_2, P_3 then:

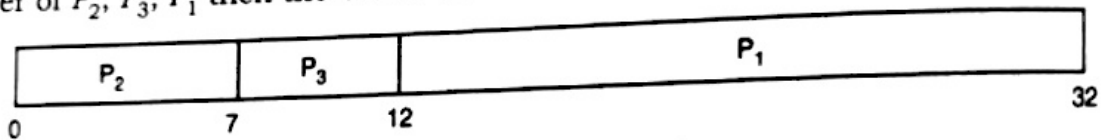
Waiting time for process $P_1 = 0$ ms

Waiting time for process $P_2 = 20$ ms

Waiting time for process $P_3 = 27$ ms

$$\therefore \text{Average waiting time} = \frac{0 + 20 + 27}{3} \text{ ms} = \frac{47}{3} \text{ ms} = 15.67 \text{ ms.}$$

(ii) If the processes P_1, P_2, P_3 arrives at the same time say at 0 ms in the system and in the order of P_2, P_3, P_1 then the **Gantt chart** will be



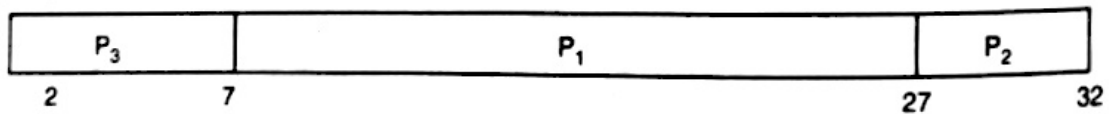
Waiting time for process $P_2 = 0$ ms

Waiting time for process $P_3 = 7$ ms

Waiting time for process $P_1 = 12$ ms

$$\text{Average waiting time} = \frac{0 + 7 + 12}{3} \text{ ms} = \frac{19}{3} \text{ ms} = 6.33 \text{ ms.}$$

(iii) If the three processes P_1, P_2 and P_3 arrives at the same time say 2 ms in the order of P_3, P_1, P_2 then the **Gantt chart** will be



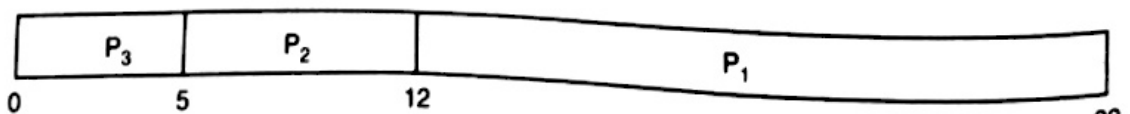
Waiting time for process $P_1 = (7 - 2)$ ms = 5 ms

Waiting time for process $P_2 = (27 - 2)$ ms = 25 ms

Waiting time for process $P_3 = 0$ ms

$$\therefore \text{Average waiting time} = \frac{5 + 25 + 0}{3} \text{ ms} = \frac{30}{3} \text{ ms} = 10 \text{ ms.}$$

(iv) If the processes arrive in the order of P_3, P_2, P_1 at the same time say 0 ms then the **Gantt chart** will be



Waiting time for process $P_1 = 12$ ms

Waiting time for process $P_2 = 5$ ms

Waiting time for process $P_3 = 0$ ms

$$\therefore \text{Average waiting time} = \frac{12 + 5 + 0}{3} \text{ ms} = \frac{17}{3} \text{ ms} = 5.67 \text{ ms.}$$

Hence, from the example it is clear that if processes of short burst time arrives firstly then the average waiting time is minimum.

So, in FCFS scheduling **order** of processes plays a major role in the measurement of average waiting time.

4.5.2 Shortest-Job-First (SJF) Scheduling Algorithm

Shortest-Job-First (SJF) scheduling is a different approach to CPU scheduling. This algorithm associates with each process the length of the latter's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the two processes have the same length of CPU burst then FCFS scheduling algorithms is followed. As scheduling is done by examining the length of next CPU burst of a process rather than its total length so it is also called shortest next CPU burst scheduling algorithm.

The key concept of this algorithm is "CPU is allocated to the process with least CPU - burst time". This algorithm is considered to be an **optimal algorithm**, as it gives the minimum average waiting time as a result.

The **disadvantage** of this algorithm is the problem to know the length of time for which CPU is needed by a process. A prediction formula may be used to predict the amount of time for which CPU may be required by a process. Let t_n be the length of n th CPU burst and let τ_{n+1} be our predicted value for the next CPU burst. Defining a parameter ' α ' that controls the relative weight of recent and past history in our prediction such that $0 \leq \alpha \leq 1$ then a formula called **exponential average** is defined as:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

where t_n contains our most recent information; and τ_n stores the past history in prediction

if $\alpha = 0$, then $\tau_{n+1} = \tau_n \Rightarrow$ recent history has no effect (current conditions are assumed to be transient)

If $\alpha = 1$, then $\tau_{n+1} = t_n \Rightarrow$ in this case only the most recent CPU burst matters.

If $\alpha = 1/2$ then recent and past history are equally weighted.

The SJF algorithm may be either preemptive or non-preemptive

The choice arises when a new process arrives in the ready queue while a previous process is executing.

A **preemptive SJF** will preempt this currently executing process and starts the execution of newly entered process while **non-preemptive SJF** will allow the currently executing process to complete its burst time without any interruption in its execution.

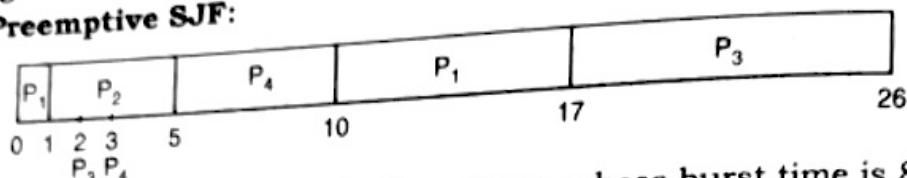
Preemptive SJF scheduling is sometimes called **Shortest-Remaining-Time-First (SRTF)** scheduling.

EXAMPLE 4.2

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Calculate Average waiting time in (i) preemptive SJF scheduling and (ii) Non-preemptive SJF scheduling.

Solution: (i) Preemptive SJF:



At $t = 0$ ms only one process P_1 is in the system, whose burst time is 8 ms; starts its execution. After 1 ms, i.e., at $t = 1$, new process P_2 (burst time = 4 ms) arrives in the ready queue. $\therefore$ Its burst time (4 ms) is less than the remaining burst time of P_1 (7 ms) so P_1 is preempted and execution of P_2 is started. Again after one ms i.e., at $t = 2$, a new process P_3 arrive in ready queue but its burst time (9 ms) is larger then the remaining burst time of currently executing process (3 ms). So P_2 is not preempted and it continuous its execution. Again after one ms at $t = 3$, new process P_4 (burst time = 5 ms) is arrived but again for the same reason P_2 is not preempted until its execution is completed. At $t = 5$ all the processes have arrive in the system. Now the processes are executed in SJF scheduling manner resulting as following:

$$\text{Waiting time for process } P_1 = 0 \text{ ms} + (10 - 1) \text{ ms} = 9 \text{ ms}$$

$$\text{Waiting time for process } P_2 = 1 \text{ ms} - 1 \text{ ms} = 0 \text{ ms}$$

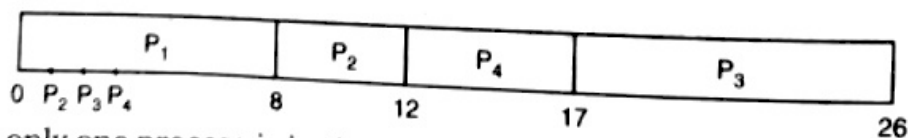
$$\text{Waiting time for process } P_3 = 17 \text{ ms} - 2 \text{ ms} = 15 \text{ ms}$$

$$\text{Waiting time for process } P_4 = 5 \text{ ms} - 3 \text{ ms} = 2 \text{ ms}$$

$$\therefore \text{Average waiting time} = \frac{9 + 0 + 15 + 2}{4} \text{ ms}$$

$$= \frac{26}{4} = 6.5 \text{ ms. Ans.}$$

(ii) Non-Preemptive SJF:



At $t = 0$ ms only one process is in the ready queue P_1 (burst time = 8 ms). Since in non-preemptive scheme when a process starts its execution then it is not preempted unless its burst time is completed. So, even P_2 , P_3 and P_4 are arrived one after the another in the duration of one ms P_1 continues its execution upto 8 ms. After this processes are executed in the accordance of SJF scheduling resulting as following:

$$\text{Waiting time for process } P_1 = 0 \text{ ms}$$

$$\text{Waiting time for process } P_2 = (8 - 1) \text{ ms} = 7 \text{ ms}$$

$$\text{Waiting time for process } P_3 = (17 - 2) \text{ ms} = 15 \text{ ms}$$

$$\text{Waiting time for process } P_4 = (12 - 3) \text{ ms} = 9 \text{ ms}$$

$$\therefore \text{Average waiting time} = \frac{0 + 7 + 15 + 9}{4} \text{ ms}$$

$$= \frac{31}{4} = 7.75 \text{ ms. Ans.}$$

4.5.3 Priority Scheduling

A priority is associated with each process, and the CPU is allocated to the process with the highest priority. *Equal-priority* processes are scheduled in FCFS order.

An SJF is simply a priority algorithm where the priority is the **inverse** of the predicted next CPU burst. The larger the CPU burst, the lower the priority and vice-versa.

Priorities are generally some fixed range of numbers such as 0 to 7. However, there is no general agreement on whether 0 is the highest or lowest priority. Some system represents low numbers to represent low priority; others use low numbers to represent higher priority.

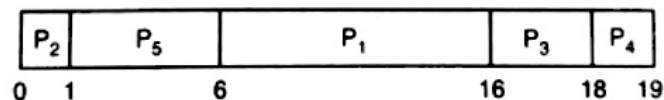
This difference can lead to confusion. In our text we use low numbers to represent high priority.

EXAMPLE 4.3 Consider the following set of processes, assumed to have arrived at time 0, in the order $P_1, P_2, \dots, P_5$, with the length of the CPU-burst time given in milliseconds:

Process	Burst time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Using priority scheduling calculate average waiting time.

Solution: Using priority scheduling, we would schedule the given processes according to the following Gantt chart:



Waiting time for process $P_1 = 6$ ms

Waiting time for process $P_2 = 0$ ms

Waiting time for process $P_3 = 16$ ms

Waiting time for process $P_4 = 18$ ms

Waiting time for process $P_5 = 1$ ms

$$\text{Avg. waiting time} = \frac{6 + 0 + 16 + 18 + 1}{5} \text{ ms} = 8.2 \text{ ms.}$$

Priority Scheduling is Either Preemptive or Non-preemptive

When a process arrives at ready queue, its priority is compared with the currently running process.

A **preemptive priority scheduling** algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the process currently running in the system. A **non-preemptive priority scheduling** algorithm will simply put the new

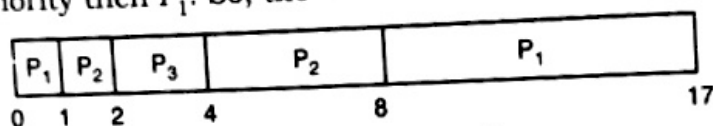
process at the head of the ready queue i.e., it does not preempt the execution of current process that is being in execution even when the priority of newly arrived process is higher than the priority of the process currently running in the system.

EXAMPLE 4.4 Preemptive Priority Scheduling

Process	Burst time	Priority	Arrival time
P_1	10	3	0
P_2	5	2	1
P_3	2	1	2

Solution: Initially we have a process P_1 (burst time = 10 ms) in the system.

After one ms, at $t = 1$ new process P_2 (5 ms) and priority 2 arrives in the systems. Since, its priority is higher than the priority of P_1 (3). So P_1 is preempted and P_2 started its execution. Again at $t = 2$, P_3 (priority = 1) arrives so P_2 is preempted for the same reason and P_3 is executed until its execution is completed. Since P_3 has got highest priority among the three processes. After the completion of P_3 , remaining part of process P_2 starts its execution as it is got higher priority than P_1 . So, the Gantt chart is as:



Waiting time for process $P_1 = 0 \text{ ms} + (8 - 1) \text{ ms} = 7 \text{ ms}$

Waiting time for process $P_2 = (4 - 2) \text{ ms} = 2 \text{ ms}$

Waiting time for process $P_3 = (2 - 2) \text{ ms} = 0 \text{ ms}$

$$\therefore \text{Average w.t.} = \frac{7 + 2 + 0}{3} \text{ ms} = 3 \text{ ms.}$$

Priorities may be assigned by the user or by the operating system, depending upon the importance of the process. A natural question that arises here is:

What will happen if the two processes having same priorities? The tie will be broken by comparing their CPU burst time. The process with shorter burst time is firstly executed and if there may be a situation that two processes having same burst time then they are executed on the basis of FCFS scheduling.

The **problem with priority** scheduling algorithm arises when it becomes the biggest cause of **starvation** of a process. If a process is in ready state but its execution is almost always preempted, due to the arrival of higher priority processes, it will starve for its execution.

Therefore, a mechanism called "**ageing**" has to be built into the system so that almost every process should get the CPU in a fixed interval of time. This can be done by increasing the priority of a low-priority process after a fixed interval of time, so that one moment of time it becomes a high-priority job as compared to others and thus, finally gets CPU for its execution.

4.5.4 Round-Robin Scheduling Algorithm

The Round-Robin (RR) scheduling algorithm is designed especially for time-sharing systems. It is similar to FCFS scheduling algorithm, but preemption is added to switch between processes. A small unit of time called a **time quantum** (or time-slice) is defined where range is kept between 10 to 100 ms.

In this algorithm Ready Queue is assumed to be a circular Queue. The CPU scheduler (short-term) goes around the ready queue, allocating the CPU to each process for a time interval of upto 1 time quantum.

To implement RR scheduling, we keep the ready queue as a FIFO Queue of the processes. New processes are added to the tail of the ready queue. The CPU scheduler picks the first process from the ready queue, sets a **timer** to interrupt after 1 time quantum, and dispatches the process.

One of the two things may happen. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then processed to the next process in the ready queue. Otherwise, if the CPU burst of the currently running process is longer than 1 time quantum, the timer will go of and will cause an interrupt to the OS. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue.

The average waiting time under Round Robin (RR) scheduling is often, quite long. If CPU burst of a process exceeds one time quantum the process is preempted and is put back in the ready queue (figure 4.3).

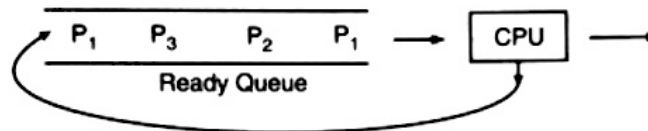


Fig. 4.3. RR scheduling

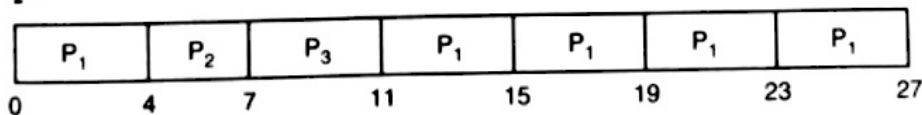
If there are N processes in the ready queue and the time quantum is ' t ', then each process gets $1/N$ of CPU time in hunks of at most ' t ' time units.

Each process must wait no longer than $(N - 1) \times t$ time units until its next time quantum.

EXAMPLE 4.5 Consider the following set of processes that arrives at time 0 ms.

Process	Burst Time
P_1	20
P_2	3
P_3	4

If we use time quantum of 4 ms then calculate the average waiting time using R-R scheduling.



Solution: According to R-R scheduling processes are executed in FCFS order. So, firstly P_1 (burst time = 20 ms) is executed but after 4 ms it is preempted and new process P_2 (burst time = 3 ms) starts its execution whose execution is completed before the time quantum then next process P_3 (burst time = 4 ms) starts its execution and finally remaining part of P_1 gets executed with time quantum of 4 ms.

Waiting time of process $P_1 = 0 \text{ ms} + (11 - 4) \text{ ms} = 7 \text{ ms}$

Waiting time for process $P_2 = 4 \text{ ms}$

Waiting time for process $P_3 = 7 \text{ ms}$

$$\therefore \text{Average waiting time} = \frac{7 + 4 + 7}{3} \text{ ms} = 6 \text{ ms.}$$